

GRAAL Lab - Manual for interact with Panda manipulator by Franka Emika

Andrea Tiranti

19 aprile 2023

1 Passwords of the machine connected to the robots

- **Ubuntu 20.04** *user*: graal, *password*: graal
- **Windows 10** *user*: graal, *password*: GR44LGR44L
- **Franka World** *user*: enrico.simetti@unige.it, *password*: graalgraalgraal

2 Prerequisite

In order to work with the robot is strongly suggested to read the official manual at <https://www.generationrobots.com/media/franka-emika-robot-handbook.pdf>. There are two ways for controlling the manipulators, through the on-line interface called Franka Desk, or using the Franka Control Interface (FCI). In the first case, is only possible to plan sequence of tasks, which are already implemented by the manufacturer. In order to control the robot through a control script implemented by the user, the FCI is mandatory together with the *real-time kernel capabilities* of the operating system. The documentation of the FCI can be found at <https://frankaemika.github.io/docs/>, is required to install the libraries in *c++* called *libfranka*. For now, in this guide we skip the use of the Desk for controlling the robot, the Franka Desk will be used only for initialize the system to work with the FCI.

3 *libfranka* installation

There are different ways to install the libraries, I strongly suggest to install from source using the guide present in the FCI documentation https://frankaemika.github.io/docs/installation_linux.html. In this section I will write down the suggested steps, especially some warnings and some insights. (TO DO) For now, the installation procedure updated by me is in the file `FRANKA_GUIDE_installation.md`, saved in the desktop of the machine connected to the robot.

4 Operating with the robot

The overall system is composed by the following items, check the correct functioning of everything listed below before using the robot:

- The *controller*, the black box under the table
- The *robotic arm*, which has:
 - The *gripper*, with a *button* on the side. If the robot is in **interaction mode**, the lights are **WHITE**, we can move around the manipulator by hands. For doing so, **the gripper's button must be pressed**.
 - A set of buttons on the top of the last joint. The buttons reproduce the functionalities of the Franka Desk Interface.

- Two buttons on the desk (for each robot):
 - **RED**: This button cut off the electricity. Therefore the joints are locked instantly. Must be always avoided, can be used only if someone is in real danger. The encoder and the mechanical limit switch can get damaged.
 - **GREY**: This button activate the FCI, if pressed the light is **WHITE** and the robot can be touched and moved by the operator. Remember to press the button of the gripper for moving it. If is no pressed the light is **BLUE**, and the robot is ready to receive commands from the controller, **STAY AWAY AND DO NOT TOUCH**.
 - **THE GREY BUTTON IS ALSO USED FOR STOP THE ROBOT DURING THE MOTION, WITHOUT ANY DANGER, USE IT ANYTIME YOU SEE MOVEMENTS THAT YOU DO NOT EXPECT.**
- Two ethernet cables, the blue one is for **ARM2** the yellow one is for **ARM1**

4.1 Turn on the system

At the GRAAL we have the complete system offered by Franka Emika, therefore we have the controller, the FCI and the libraries installed. In this case the manipulator must be turned on through the controller. The switch is the one in Figure 1. After pressed the button the lights of the manipulator become **flashing** and **yellow**. After the system is ready, the lights stop flashing.



Figura 1: Switch of the controller

Once the controller is switched on, you can connect to the robot using the Franka Desk. The Desk can be accessed through internet by typing <https://robot-ip/desk/>. At the GRAAL the machine connected to the robot is already configured, therefore the internet access pages are saved as Bookmarks in the Mozilla Firefox Browser. The Bookmarks are two, one for each robot, and named as **PANDA1** and **PANDA2**, in order to recognize the two manipulators, **the numbers 1 and 2 are the same of the ones written in the post-it on the desk!**. **NB:** In the MATLAB code used by the students the **LEFT ARM** corresponds to **ARM2** the **RIGHT ARM** corresponds to **ARM1**. After accessed the *Franka Desk* you should see the following page 2.

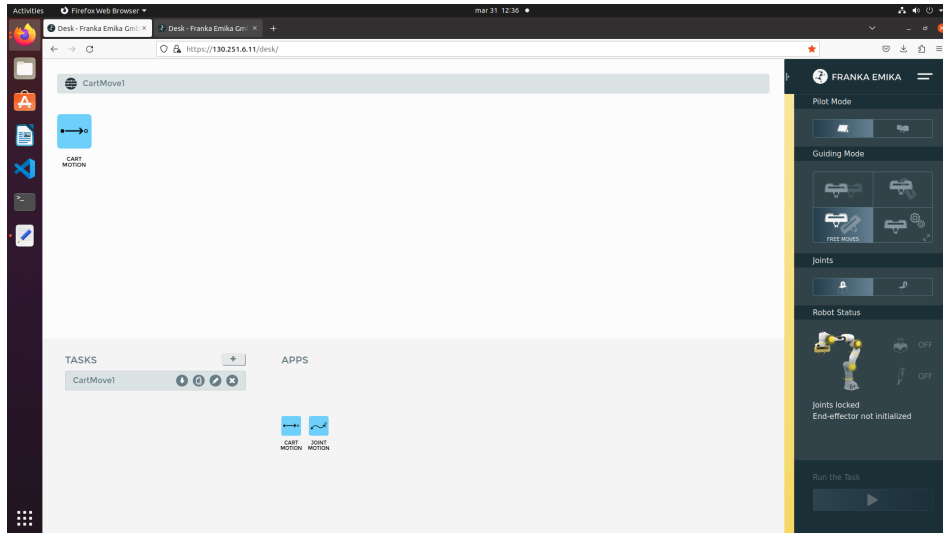


Figura 2: *Franka Desk* after the switching on

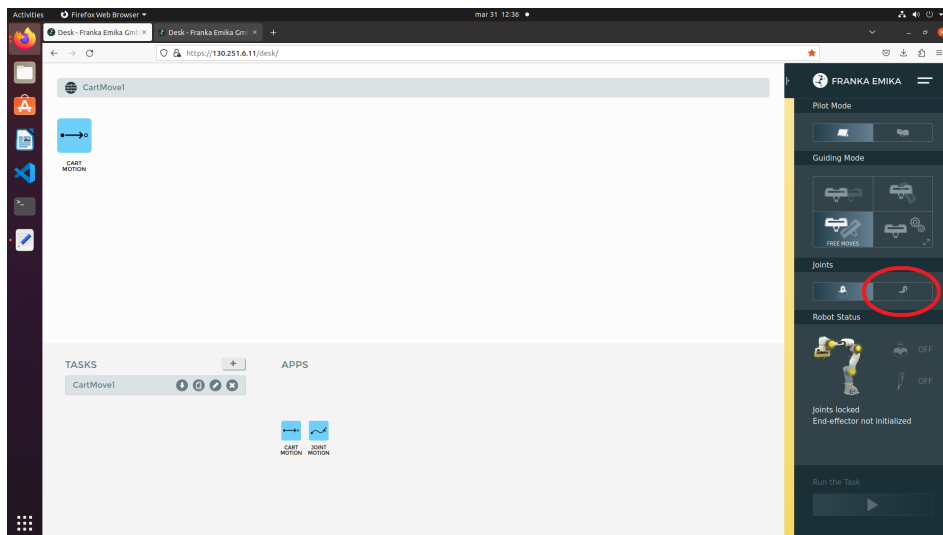


Figura 3: Unlock the joints

4.2 Activate the Franka Control Interface

After accessed the *Franka Desk*, the first thing to do is to **UNLOCK THE JOINTS** by clicking on the **BUTTON** depicted in Figure 3. After finished the procedure the lights of the robot became **WHITE**. Now we need to activate the FCI by clicking *Activate FCI* on the drop-down menu on the top left corner. **N.B.** Once the lights are **WHITE** you can already play with the robot by moving it by the end-effector, **remember to touch the button of the end-effector while moving**. When you are ready to control the robot through the FCI, the security button (the grey one) must be **disabled!** After that, the lights became **BLUE**.

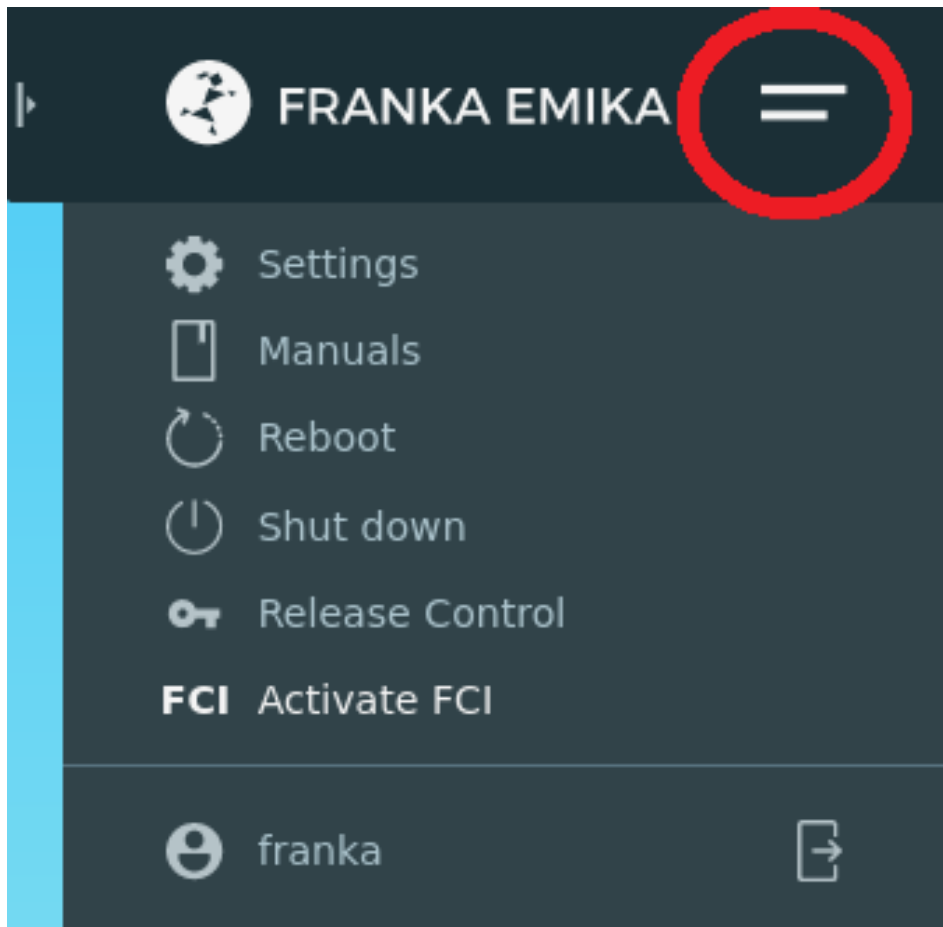


Figura 4: How to activate FCI

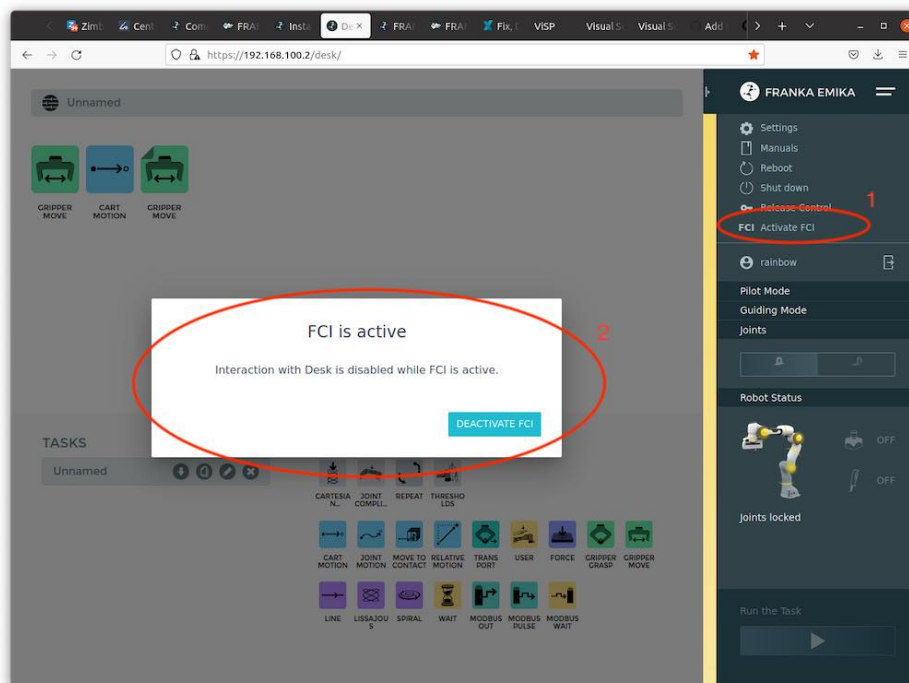


Figura 5: FCI ACTIVATED

4.3 Use the pybullet simulator

Is fundamental to test your MATLAB script BEFORE using the robot, hence a simulator is provided. The simulator is inside the folder *pybullet simulation* on the *Desktop*. In order to launch the simulator digit in a shell:

```
$ cd Desktop/pybullet_simulation
$ python3 franka_panda_simulation.py
```

The following lines of code must be at the beginning of every control script you write in Matlab (the example is for two manipulators, if using one I suggest to keep the same UDP port numbers):

```
real_robot = true;
% UDP Connection with Franka Interface
if real_robot == true
    hudprLeft = dsp.UDPReceiver('LocalIPPort',1501,'MaximumMessageLength',255);
    hudprRight = dsp.UDPReceiver('LocalIPPort',1503,'MaximumMessageLength',255);
    hudpsLeft = dsp.UDPSender('RemoteIPPort',1500);
    hudpsLeft.RemoteIPAddress = '127.0.0.1';
    hudpsRight = dsp.UDPSender('RemoteIPPort',1502);
    hudpsRight.RemoteIPAddress = '127.0.0.1';
else
    hudps = dsp.UDPSender('RemoteIPPort',1505);
    hudps.RemoteIPAddress = '127.0.0.1';
end
```

The default settings for the UDP connection on the GRAAL machine is the following, sender port indicates the UDP Port where we send JOINT VELOCITIES from MATLAB to the UDP joint velocities interface, while the receiver port is the UDP Port where we read the feedback from the robot, the feedback are the joint positions:

- 1501 is the UDP Receiver port number for ARM2.
- 1503 is the UDP Receiver port number for ARM1.
- 1500 is the UDP Sender port number for ARM2.
- 1502 is the UDP Sender port number for ARM1.
- 1505 is the port Number of the simulator.

In this way by setting the variable *real_robot* to *True* the UDP connection to the manipulators is established, otherwise the matlab script is connected to the simulator.

4.4 Run a MATLAB script

NOW YOU HAVE TO REBOOT THE SYSTEM AND LOAD THE THE REAL TIME PATCH OF THE LINUX KERNEL. On the boot menu choose *Advanced settings for ubuntu* and then *ubuntu-rt-44 patch*. The procedure to test a Matlab script is the following (the example is for two manipulators):

- Make sure that the FCI is activated and the lights are **BLUE**
- Run the UDP Joint Velocities Interface. For connecting with ARM2 digit in a shell:

```
$ cd libfranka/build/examples/
$ ./udp_joint_vel_control 130.251.6.12 1500 1501
```

- For connecting with ARM1 digit in a shell:

```
$ cd libfranka/build/examples/
$ ./udp_joint_vel_control 130.251.6.11 1502 1503
```

- Now the UDP JOINT VELOCITIES INTERFACE is launched and connected (must appear a print on the shell that inform that the socket has been created successfully).
- Press ENTER to move the robot to the HOME POSITION. **WARNING!! ALL PEOPLE AWAY FROM THE ROBOT.**
- Run your MATLAB script.
- **IF ANYTHING GO WRONG PRESS THE GREY BUTTON AND STOP EVERYTHING!**
- **REMEMBER TO RESTART THE UDP JOINT VEL INTERFACE EACH TIME YOU RUN A NEW MATLAB SCRIPT.** (kill the process and then re-run the interface as before).
- IMPORTANT REMARKS and SAFETY BEHAVIOURS
 - Se per qualche motivo il **movimento del robot è stato interrotto** dall'utente schiacciando il pulsante grigio, fermando l'interfaccia UDP o fermando lo script matlab. **Bisogna resettare gli encoder del manipolatore** per rilanciare nuovamente l'interfaccia UDP. In tal caso, se si prova a lanciare l'interfaccia di controllo (i.e. the udp joint velocities control), il messaggio "Cartesian Reflex" viene stampata sulla shell. **Per resettare gli encoder basta premere il pulsante grigio e spostare MANUALMENTE il robot** (basta un piccolo movimento).
 - Se MATLAB si chiude inaspettatamente premere sempre il pulsante grigio. Se il robot continua a muoversi anche dopo la fine dello script premere sempre il pulsante grigio.
 - Quando rilanciate lo script di interfaccia *udp* e premerete *Enter*, i robot tornano in *home position*. **FARE ATTENZIONE CHE I ROBOT NON SIANO VICINI TRA LORO E POSSANO COLLIDERE!!!**. In generale, lanciare le interfacce una alla volta, così i robot non vanno in home position simultaneamente. In caso non siete sicuri spostate manualmente i robot, così da allontanarli.

4.5 Run the C++ examples

I already prepared some nice examples, for now we have two possible experiments:

1. Controllo di impedenza ai giunti, per far disegnare al robot un cerchio. VIENE STAMPATO IL TORQUE COMANDATO e quello MISURATO.
2. Controllo di impedenza cartesiana. Il robot rimane fermo lo si può toccare liberamente per verificare che torna in posizione. Si può comprendere cosa significa il controllo in impedenza e cosa implica la ridondanza del sistema.

NOW YOU HAVE TO REBOOT THE SYSTEM AND LOAD THE THE REAL TIME PATCH OF THE LINUX KERNEL. On the boot menu choose *Advanced settings for ubuntu* and then *rt-44 patch* (something similar, do not choose the *recovery mode*). The procedure to test a C++ script is the following (the examples are for **one** manipulators):

- Make sure that the FCI is activated and the lights are **BLUE**
- Run the example 1:

```
$ cd libfranka/build/examples/
$ ./cartesian_impedance_control <robot-ip>
$ Press Enter (wait for the shell to print the output of the initialization)
```

- Run the example 2:

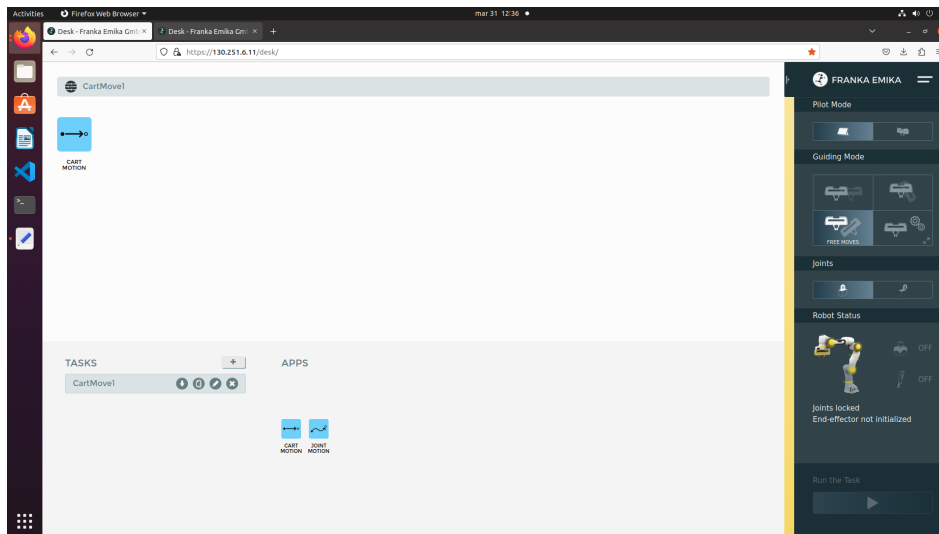


Figura 6: Overview *Franka Desk*

```
$ cd libfranka/build/examples/
$ ./joint_impedance_control <robot-ip>
$ Press Enter (wait for the shell to print the output of the initialization)
```

In order to compile and build your code in c++:

```
$ cd libfranka/examples/
$ <your_code>.cpp
$ Add the name <your_code> in the CMakeList.txt (in the same folder).
$ cd ..
$ cd build/examples/
$ cmake --build .
```

4.6 Run the Simulink examples

I have the TPIK algorithm in simulink, and other examples. Is possible to design control schemes in Simulink using special blocks given by the manufacturers. TODO

4.7 Turn off the system

- If the lights are **BLUE** press the **GREY BUTTON**, if the lights are **WHITE** skip.
- When the lights are **WHITE**, deactivate the FCI from the *Franka Desk* web page. By clicking on the **DEACTIVATE FCI** button
- **LOCK THE JOINTS** by clicking on the appropriate button. The lights should turn **YELLOW**.
- Select **TURN OFF THE SYSTEM** from the drop-down menu on the top left corner 5.
- After the procedure is finished the *Franka Desk* will notify the correct shutdown of the system
- **SWITCH OFF THE CONTROLLER!** Even if the lights are off the controller is still switched on.

5 Matlab Demos

For now, we have three examples in Matlab, that can also be used as exercises for the students.

- The first example implements the *Inverse Kinematic* algorithm on Matlab. Is suitable for students of the course of *Modelling and Control of Manipulators*. The path to the file is the following: `\home\graal\Desktop\demo_matlab\ex1\inverse_kinematic.m`
- The second example implements the *bimanual rigid grasp task* performed with the two manipulators as a *bimanual system*. Suitable for the course of *Cooperative Robotics*. The path to the folder is the following: `home\graal\Desktop\demo_matlab\ex2` The folder contain the *main.m* file for launching the example, and the MATLAB functions, used in the implementation of the Task Priority Inverse Kinematic algorithm.
- The last example implements also the *bimanual rigid grasp task*, now the mission is structured in more phases, so the two arms move the object in a sequence of five position iteratively. The example ends after three iterations. Now, the robot are *two separate systems* (simulated in MATLAB) and the cooperation is achieved thanks to a specific *cooperation policy*. This example is suitable for the course of *Cooperative Robotics*. The path to the folder is the following: `home\graal\Desktop\demo_matlab\ex3` The folder contain the *main.m* file for launching the example, and the MATLAB functions, used to implement the TPIK.

6 Implement your own code in C++

TODO

7 Implement your own code in Matlab

TODO